

1.1(g)

The problem can be written as a QP

$$\min_x \frac{1}{2}x^T Qx - b^T x \quad x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad Q = \begin{bmatrix} 1 & 0 \\ 0 & \gamma \end{bmatrix} \quad b = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Since Q is positive definite, the optimal solution is

$$x^* = Q^{-1}b = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

The constant-step plots use $\eta = 0.15$. The level sets satisfy

$$x_1^2 + \gamma x_2^2 = c$$

For $\gamma = 10$, these are narrow ellipses, so the curvature in the x_2 direction is much larger than in the x_1 direction. This causes zig-zagging for both exact line search and constant-step gradient descent.

For constant-step gradient descent,

$$x^{(k+1)} = (I - \eta Q)x^{(k)}$$

so

$$x_2^{(k+1)} = (1 - \eta\gamma)x_2^{(k)}$$

Zig-zagging begins when $1 - \eta\gamma < 0$. Convergence requires $|1 - \eta\gamma| < 1$, which gives $0 < \eta < 2/\gamma$. Then, for $\gamma = 10$, $\eta = 0.15$ zig-zags but still converges, while steps larger than 0.2 diverge. For $\gamma = 1$, the level sets are circular and exact line search reaches the optimum in one step.

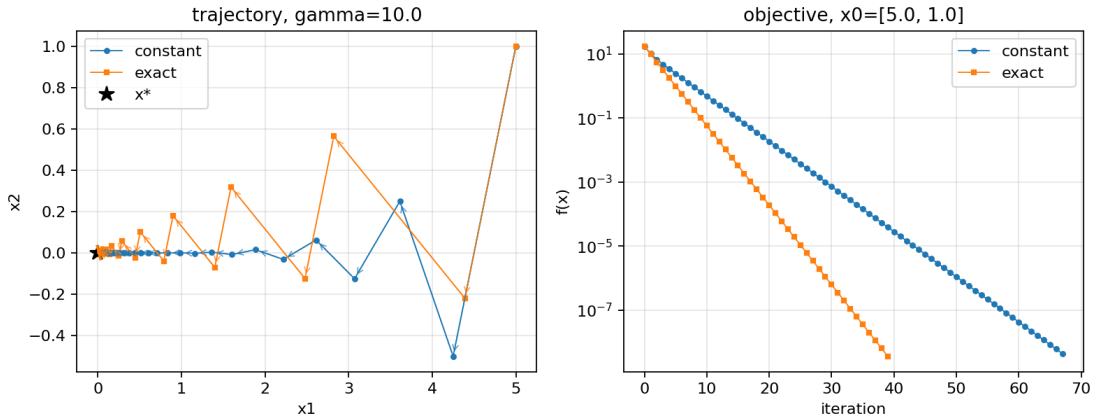


Figure 1: $\gamma = 10$, $x^{(0)} = (5, 1)$.

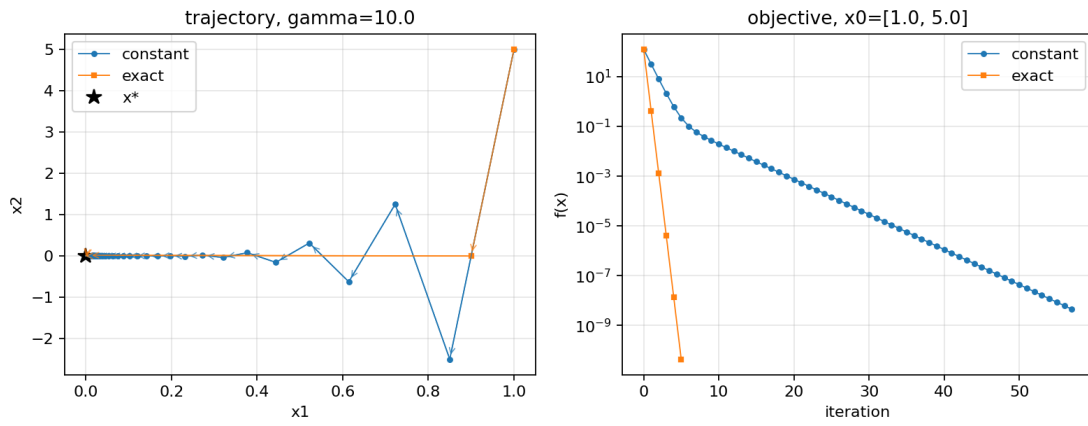


Figure 2: $\gamma = 10$, $x^{(0)} = (1, 5)$.

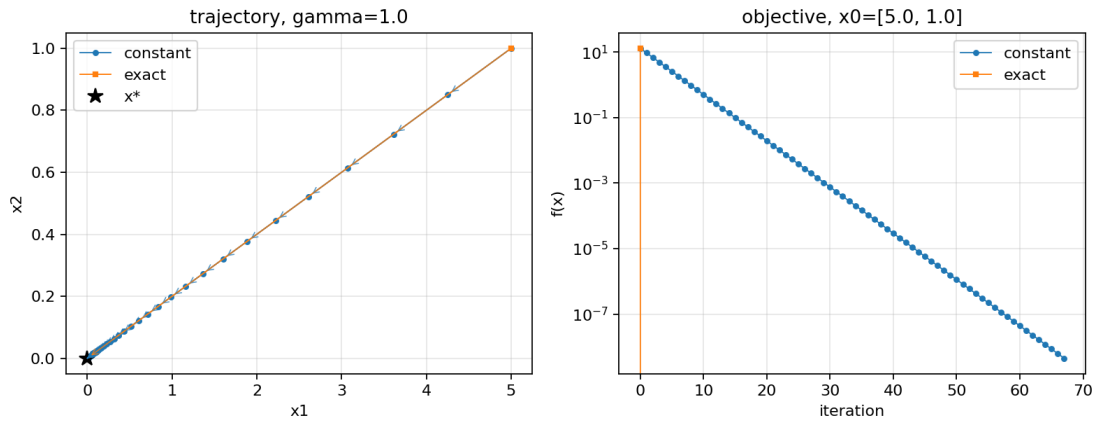


Figure 3: $\gamma = 1$, $x^{(0)} = (5, 1)$.

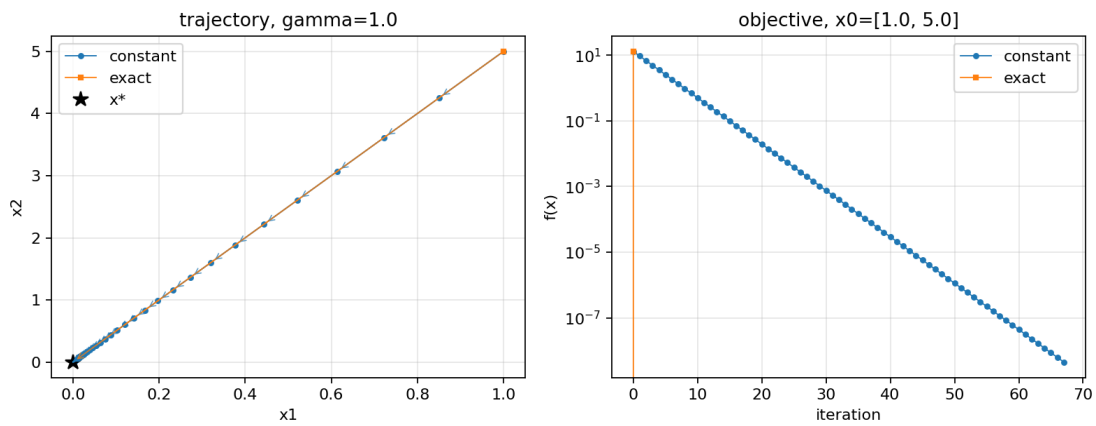


Figure 4: $\gamma = 1$, $x^{(0)} = (1, 5)$.

Code:

```
1 def f(x, q, b):
2     return 0.5 * x.T @ q @ x - b.T @ x
```

```

3
4 def grad_f(x, q, b):
5     return q @ x - b
6
7 def gd_constant(x0, q, b, eta):
8     x = x0.astype(float).copy()
9     traj = [x.copy()]
10    vals = [f(x, q, b)]
11    tol = 1e-4
12
13    while np.linalg.norm(grad_f(x, q, b)) > tol:
14        g = grad_f(x, q, b)
15        x = x - eta * g
16        traj.append(x.copy())
17        vals.append(f(x, q, b))
18
19    return np.array(traj), np.array(vals)
20
21 def gd_exact_line_search(x0, q, b):
22     x = x0.astype(float).copy()
23     traj = [x.copy()]
24     vals = [f(x, q, b)]
25     tol = 1e-4
26
27     while np.linalg.norm(grad_f(x, q, b)) > tol:
28         d = -grad_f(x, q, b)
29         eta = (d.T @ d) / (d.T @ q @ d)
30         x = x + eta * d
31         traj.append(x.copy())
32         vals.append(f(x, q, b))
33
34     return np.array(traj), np.array(vals)
35
36 def run(gamma, x0):
37     q = np.diag([1.0, gamma])
38     b = np.zeros(2)
39     eta_const = 0.15
40     traj_const, vals_const = gd_constant(x0, q, b, eta_const)
41     traj_ls, vals_ls = gd_exact_line_search(x0, q, b)

```

1.2(b)

Using constant-step gradient descent with step size $\eta = 10^{-5}$.

The terminal output from the implementation is:

```
eta_const = 1.000000e-05
iterations = 279802
final objective = -27.052877
LQR cost J(u*) = 2.947123
```

Code:

```
1 T = 20
2 A = np.array([[1.0, 1.0], [0.0, 1.0]])
3 B = np.array([[0.0], [1.0]])
4 Q = np.eye(2)
5 R = np.eye(1)
6 Q_T = 10.0 * np.eye(2)
7 x0 = np.array([1.0, 0.0])
8
9 n = A.shape[0]
10 m = B.shape[1]
11 u0 = np.zeros(m * T)
12
13 nu = np.zeros((n * T, n))
14 V = np.zeros((n * T, m * T))
15
16 a_powers = [np.eye(n)]
17 for _ in range(T):
18     a_powers.append(a_powers[-1] @ A)
19
20 for t in range(T):
21     nu[t*n:(t+1)*n, :] = a_powers[t + 1]
22     for k in range(t + 1):
23         V[t*n:(t+1)*n, k*m:(k+1)*m] = a_powers[t - k] @ B
24
25 H = np.zeros((n * T, n * T))
26 for t in range(T - 1):
27     H[t*n:(t+1)*n, t*n:(t+1)*n] = Q
28 H[(T - 1)*n:T*n, (T - 1)*n:T*n] = Q_T
29
30 N = np.zeros((m * T, m * T))
31 for t in range(T):
32     N[t*m:(t+1)*m, t*m:(t+1)*m] = R
33
34 Q_tilde = 2.0 * (V.T @ H @ V + N)
35 b_tilde = -2.0 * V.T @ H @ nu @ x0
36
37 eta_const = 1e-5
38 traj_const, vals_const = gd_constant(u0, Q_tilde, b_tilde, eta_const)
39
40 u_star = traj_const[-1]
41 x_stack = nu @ x0 + V @ u_star
42 J_lqr = x0.T @ Q @ x0
43 for t in range(T - 1):
44     x_t = x_stack[t*n:(t+1)*n]
45     u_t = u_star[t*m:(t+1)*m]
46     J_lqr += x_t.T @ Q @ x_t + u_t.T @ R @ u_t
47 x_T = x_stack[(T - 1)*n:T*n]
48 J_lqr += x_T.T @ Q_T @ x_T
```